Good. This is the correct way to consolidate knowledge.

Below is a **Phase 1 Concept Reinforcement Sheet** — all questions I asked you + corrected answers + additional production-level questions.

Read this carefully. This is DevOps mindset foundation.

✅ Phase 1 — DevOps Foundations (Q&A Consolidation)

1️⃣ Why is a health check endpoint important?

Correct Answer:

A production health check must verify:

- The process is running
- Critical dependencies (DB, cache, message broker) are reachable
- The service is capable of serving traffic

It is used by:

- Load balancers
- Kubernetes
- Monitoring systems

2️⃣ Difference Between Liveness and Readiness

Liveness

- Checks if the container process is alive
- If failed → Kubernetes restarts container

Readiness

- Checks if the container can serve traffic
- If failed → Kubernetes stops routing traffic (no restart)

3️⃣ If DB goes down temporarily, which should fail?

Answer:

Readiness should fail.

Liveness should pass.

4️⃣ Why must scalable systems be stateless?

Correct Answer:

Because load balancers distribute requests across replicas.

State stored in one container's memory is not available in others, causing inconsistency.

State must live in:

- Database
- Redis
- External storage

5️⃣ Should DB URL be hardcoded?

Answer:

No.

Use environment variables.

Reason:

- Different environments require different configs
- Secure secret handling
- Container portability
- Cloud-native compatibility

6️⃣ Why separate `app.js` and `server.js`?

Correct Answer:

To separate:

- Application definition (routes, middleware)
- Server lifecycle (port binding, shutdown)

Benefits:

- Easier testing
- Better lifecycle control
- Clean architecture
- Cloud compatibility

7️⃣ Why handle SIGTERM in production?

Answer:

When Kubernetes or Docker stops a container, it sends `SIGTERM`.

Without graceful shutdown:

- In-flight requests are dropped
- Users see errors
- System reliability decreases

8️⃣ Why structured logging instead of console strings?

Correct Answer:

Structured logs:

- Machine-readable
- Filterable
- Searchable
- Aggregatable

Required for:

- ELK stack
- Datadog
- Azure Monitor
- Production debugging

9️⃣ Why log inside `res.on("finish")`?

Answer:

Because only then:

- Final status code is known
- Response time can be calculated
- Async operations have completed

🔟 Should `/version` be under `/health`?

Answer:

No.

Health = infrastructure check

Version = deployment verification

1️⃣1️⃣ Why is `/version` useful in rolling deployments?

Because during rolling updates:

- Multiple versions run simultaneously
- You can confirm traffic is hitting new pods
- You can verify load balancing behavior

🔥 Now Additional Critical Phase 1 Questions

These are production-grade questions you must understand.

1️⃣2️⃣ Why should containers not run as root?

Because:

- Security risk
- If compromised, attacker gains full privileges
- Violates container best practices

1️⃣3️⃣ Why should we avoid using `latest` tag in Docker?

Because:

- Non-deterministic builds
- Impossible to guarantee which version is deployed
- Harder rollback
- Breaks CI/CD reliability

Always use:

- Semantic versioning
- Explicit image tags

1️⃣4️⃣ Why should health endpoints be lightweight?

Because:

- Kubernetes hits them frequently
- Load balancers hit them frequently
- Must not perform heavy DB queries
- Must respond fast

1️⃣5️⃣ What happens if liveness probe is misconfigured?

If it fails incorrectly:

- Kubernetes keeps restarting healthy containers
- Creates restart loop
- Causes downtime

1️⃣6️⃣ Why must logs go to stdout inside containers?

Because:

- Container runtime captures stdout
- Log collectors scrape container logs
- File-based logs break in ephemeral containers

Never log to files inside containers unless explicitly mounted.

1️⃣7️⃣ Why is graceful shutdown important during scaling down?

During scaling down:

- Kubernetes removes pods
- If no graceful shutdown:
  - Active requests fail
  - Users get 502 errors

1️⃣8️⃣ Why is environment-based configuration essential for CI/CD?

Because:

- Same image runs in all environments
- Only environment variables change
- Promotes immutable infrastructure principle

1️⃣9️⃣ What is immutable infrastructure?

Infrastructure where:

- You don't modify running containers
- You deploy new versions instead of patching old ones
- Rollbacks are done via redeployment

2️⃣0️⃣ Why is stateless design required for horizontal scaling?

Because:

- Horizontal scaling = adding replicas

- Load balancer distributes traffic
- In-memory state breaks consistency

🧠 Now One Critical Question For You

If we containerize this app and later deploy in Kubernetes:

Should the container expose:

- A fixed hardcoded port (like 5000)
- Or use environment-defined port?

Answer and explain.

Think cloud-native.